Functions

If a group of statements is repeatedly required, then it is not recommended to write these statements every time separately. We have to define these statements as a single unit and we can call that unit any number of times based on our requirement without rewriting. This unit is nothing but function.

The main advantage of functions is code Reusability.

Python supports 2 types of functions

1) Built in Functions

2) User Defined Functions

1) Built in Functions:

The functions which are coming along with Python software automatically, are called built in functions or pre-defined functions.
Eg:
id()
type()
input()
eval()
etc…….

2) User Defined Functions:

The functions which are developed by programmer explicitly according to business requirements, are called user defined functions.

Syntax to Create User defined Functions:

```
def function_name(parameters):
    """ doc string"""
    ---- -----
    return value
```

Note:

While creating functions we can use 2 keywords

1) def (mandatory)
2) return (optional)

QUES

Write a function to print Hello.

```
def wish():
    print("Hello Good Morning")

wish()
```

Parameters

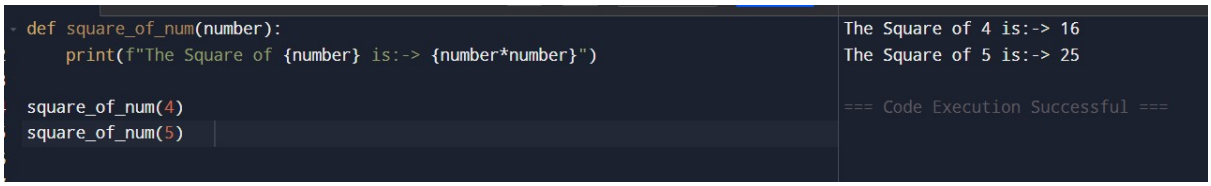
Parameters are inputs to the function. If a function contains parameters, then at the time of calling, compulsory we should provide values otherwise, otherwise we will get error.

```
def wish(name):
    print(f"Hello {name} Good Morning")

wish("Prateek")
wish("Pankaj")
```

QUES

Write a function to take number as input and print its square value

```
def square_of_num(number):
    print(f"The Square of {number} is:-> {number*number}")

square_of_num(4)
square_of_num(5)
```



Return Statement:

Function can take input values as parameters and executes business logic, and returns output to the caller with return statement.

QUES

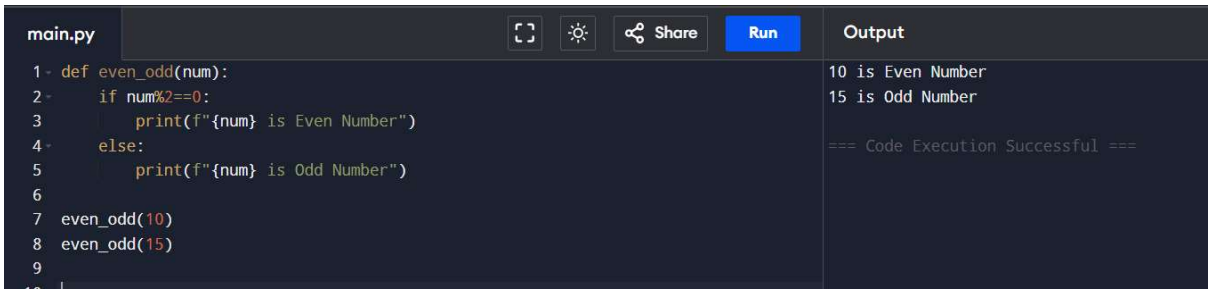
Write a Function to accept 2 Numbers as Input and return Sum

```
def add(x,y):
    return x+y

result=add(10,20)
print("The sum is",result)
print("The sum is",add(100,200))
```

QUES

Write a Function to check whether the given Number is Even OR Odd?

```
def even_odd(num):
    if num%2==0:
        print(f"{num} is Even Number")
    else:
        print(f"{num} is Odd Number")

even_odd(10)
even_odd(15)
```



QUES

Write a Function to find Factorial of given Number?

```
def fact(num):
    result=1
    while num>=1:
        result=result*num
        num=num-1
    return result

for i in range(1,6):
    print(f"The Factorial of {i} is :-> {fact(i)}")
```

Returning Multiple Values from a Function:

In other languages like C, C++ and Java, function can return almost one value. But in Python, a function can return any number of values.

```
def calc(a,b):
    sum=a+b
    sub=a-b
    mul=a*b
    div=a/b
    return sum,sub,mul,div

a, b, c, d=calc(100,50)

print(a)
print(b)
print(c)
print(d)

t=calc(100,50)
print("The Results are")
for i in t:
    print(i)
```

main.py

```python
def calc(a,b):
    sum=a+b
    sub=a-b
    mul=a*b
    div=a/b
    return sum,sub,mul,div

a, b, c, d=calc(100,50)

print(a)
print(b)
print(c)
print(d)

t=calc(100,50)
print("The Results are")
for i in t:
    print(i)
```

Output

```
150
50
5000
2.0
The Results are
150
50
5000
2.0

=== Code Execution Successful ===
```

Types of Arguments

There are 4 types are actual arguments are allowed in Python.

1) Positional Arguments

2) Keyword Arguments

3) Default Arguments

4) Variable Length Arguments

1) **Positional Arguments:**

- **These are the arguments passed to function in correct positional order.**

```
def sub(a, b):
    print(a-b)

sub(100, 200)
sub(200, 100)
```

- **The number of arguments and position of arguments must be matched.**
  If we change the order, then result may be changed.

- **If we change the number of arguments, then we will get error.**

2) **Keyword Arguments:**
   We can pass argument values by keyword i.e by parameter name.

a **keyword argument** (also called a named argument) is a way to pass values to a function by explicitly stating the parameter name. This makes the code more readable and allows you to pass arguments in any order.

```python
def create_profile(name, age, city):
    print(f"Name: {name}, Age: {age}, City: {city}")

# Calling with keyword arguments (order doesn't matter)
create_profile(age=25, city="New York", name="Alice")
```

Note: We can use both positional and keyword arguments simultaneously. But first we have to take positional arguments and then keyword arguments, otherwise we will get syntax error.

```python
def describe_pet(animal_type, name, age):
    print(f"I have a {animal_type} named {name} who is {age} years old.")

# CORRECT: Positional first, then Keywords
describe_pet("Dog", age=5, name="Buddy")
```

```python
# INVALID: Keyword followed by Positional
describe_pet(animal_type="Dog", "Buddy", age=5)
# Result: SyntaxError: positional argument follows keyword argument
```

3) Default Arguments:

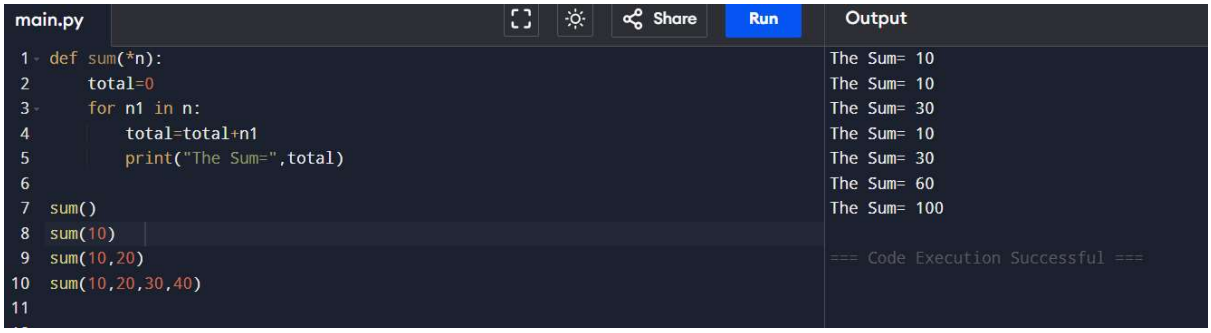
Sometimes we can provide default values for our positional arguments.

```python
# 'message' has a default value, 'name' does not
def greet(name, message="Hello"):
    print(f"{message}, {name}!")

# 1. Using the default
greet("Alice")           # Output: Hello, Alice!

# 2. Overriding the default
greet("Bob", "Hi")       # Output: Hi, Bob!
```

4) Variable Length Arguments:

- **Sometimes we can pass variable number of arguments to our function, such type of arguments are called variable length arguments.**
- **We can declare a variable length argument with * symbol as follows**
- **def f1(*n):**
- **We can call this function by passing any number of arguments including zero number.**
- **Internally all these values represented in the form of tuple.**

```
def sum(*n):
   total=0
   for n1 in n:
      total=total+n1
      print("The Sum=",total)

sum()
sum(10)
sum(10,20)
sum(10,20,30,40)
```



Note:

We can declare key word variable length arguments also.

- **For this we have to use **.**
- **def f1(**n):**
- **We can call this function by passing any number of keyword arguments. Internally these keyword arguments will be stored inside a dictionary.**

```
def  display(**kwargs):
   for k,v in kwargs.items():
      print(f"{k} = {v}")
#-----------------------------------------------
display(n1=10,n2=20,n3=30)
display(rno=100,name="Prateek",marks=70,subject="Python")
```

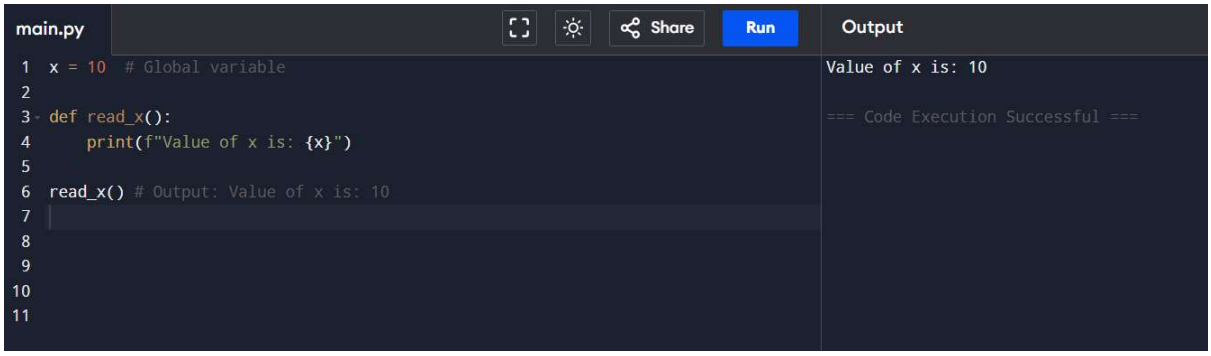
Types of Variables Python supports 2 types of variables.

1) Global Variables

2) Local Variables

1) **Global Variables**
   - **The variables which are declared outside of function are called global variables.**
   - **These variables can be accessed in all functions of that module.**

```
x = 10  # Global variable

def read_x():
    print(f"Value of x is: {x}")

read_x() # Output: Value of x is: 10
```



2) **Local Variables:**

- **The variables which are declared inside a function are called local variables.**
- **Local variables are available only for the function in which we declared it.**

  i.e from outside of function we cannot access.

```
def f1():
    a=10
    print(a)   # valid

def f2():
    print(a)    #invalid

f1()
f2()
```

NameError: name 'a' is not defined

Recursive Functions

A function that calls itself is known as Recursive Function.

```
def factorial(n):
    # Base Case: factorial of 1 is 1
    if n == 0:
        return 1
    # Recursive Step: n * factorial of (n-1)
    else:
        return n * factorial(n - 1)

print(factorial(5)) # Output: 120
```

Recursion is a programming technique where a function calls itself either directly or indirectly to solve a problem by breaking it into smaller, simpler sub problems.

Anonymous Functions:

- Sometimes we can declare a function without any name, such type of nameless functions are called anonymous functions or lambda functions.
- The main purpose of anonymous function is just for instant use (i.e for one time usage)

Syntax of lambda Function:

lambda argument_list : expression

QUES

Write a Program to create a Lambda Function to find Square of given Number?

s=lambda n:n*n

print("The Square of 4 is :",s(4))

print("The Square of 5 is :",s(5))

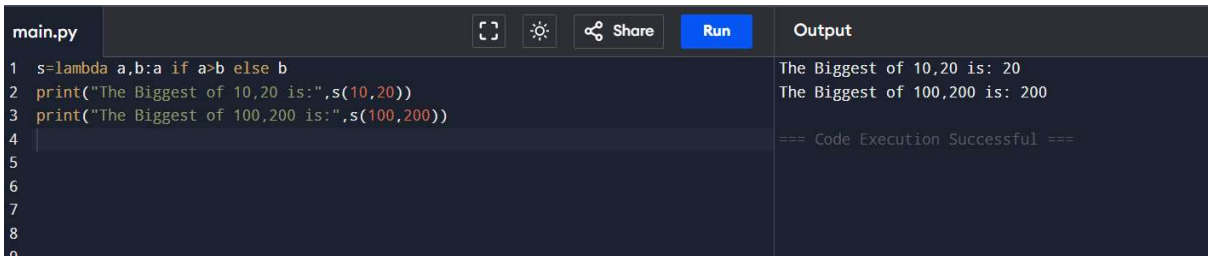
QUES

Lambda Function to find biggest of given Values

s=lambda a,b:a if a>b else b

print("The Biggest of 10,20 is:",s(10,20))

print("The Biggest of 100,200 is:",s(100,200))



filter() Function:

- **We can use filter() function to filter values from the given sequence based on some condition.**
- **filter(function, sequence)**
- **Where Function Argument is responsible to perform conditional check Sequence can be List OR Tuple OR String.**

Program to filter only Even Numbers from the List by using filter() Function?

➔ function

```
def isEven(x):
    if x%2==0:
        return True
    else:
        return False
```

➔ Sequence

l=[0,5,10,15,20,25,30]

l1=list(filter(isEven, l))

print(l1) #[0,10,20,30]

With Lambda Function:

```
l=[0,5,10,15,20,25,30]
l1=list(filter(lambda x:x%2==0,l))
print(l1)   #[0,10,20,30]
l2=list(filter(lambda x:x%2!=0,l))
print(l2) #[5,15,25]
```

map() Function:

- **For every element present in the given sequence, apply some functionality and generate new element with the required modification. For this requirement we should go for map() function.**

- **Eg: For every element present in the list perform double and generate new list of doubles.**

- **Syntax: map(function, sequence)**
- **The function can be applied on each element of sequence and generates new sequence.**

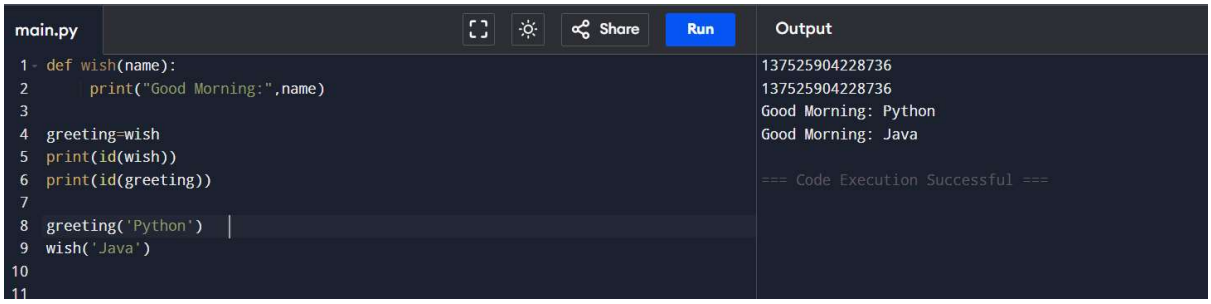
```
l=[1,2,3,4,5]

def doubleIt(x):
    return 2*x

l1=list(map(doubleIt,l))
print(l1)
```

Function Aliasing:

For the existing function we can give another name, which is nothing but function aliasing.

```
def wish(name):
    print("Good Morning:",name)

greeting=wish
print(id(wish))
print(id(greeting))

greeting('Python')
```

wish('Java')



Nested Functions:

We can declare a function inside another function, such type of functions are called Nested functions.

```
def parent_function(person_name):
    # This is the nested (inner) function
    def child_function():
        return f"Hello, {person_name}!"

    # The parent calls the child and gets the result
    message = child_function()
    print(message)

# Calling the outer function
parent_function("Alice")
```

```python
def parent_function(person_name):
    # This is the nested (inner) function
    def child_function():
        return f"Hello, {person_name}!"

    # The parent calls the child and gets the result
    message = child_function()
    print(message)

# Calling the outer function
parent_function("Alice")
```

```
Hello, Alice!

=== Code Execution Successful ===
```

Learn More

```
1)  def outer():
2)      print("outer function started")
3)      def inner():
4)          print("inner function execution")
5)      print("outer function returning inner function")
6)      return inner
7)  f1=outer()
8)  f1()
9)  f1()
10) f1()
```

Output:

```
outer function started
outer function returning inner function
inner function execution
inner function execution
inner function execution
```

Explanation:

1. The Call to `outer()` (Line 7)

When you run `f1 = outer()`, the following occurs:

- **Line 2**: Prints `"outer function started"`.
- **Line 3-4**: The `inner` function is defined in memory (but **not** executed yet).
- **Line 5**: Prints `"outer function returning inner function"`.
- **Line 6**: The `outer` function finishes and **returns the memory address** of the `inner` function.

- **Line 7**: The variable `f1` now stores that `inner` function.

2. The Calls to `f1()` (Lines 8, 9, and 10)

Since `f1` now refers to the `inner` function, every time you call `f1()`, it executes only the code inside `inner`:

- **Line 8**: Executes `inner()`, printing `"inner function execution"`.
- **Line 9**: Executes `inner()` again, printing `"inner function execution"`.
- **Line 10**: Executes `inner()` a third time, printing `"inner function execution"`.